

Lab 3 – Batch jobs with SLURM

Array/parametric jobs

Using **--array** creates an **array job** – multiple instances of the same job (with exactly the same parameters):

```
$ sbatch -N 1 --ntasks-per-node=1 -p plgrid --array=1-4 test.sh
```

This will create an array job composed of 4 jobs. This can be verified by using the **squeue** command. Each job is assigned a unique id available through the **SLURM_ARRAY_TASK_ID** environment variable:

```
$ echo $SLURM_ARRAY_TASK_ID
```

Array jobs are used for parallel processing. The **SLURM_ARRAY_TASK_ID** variable can be used in the job script to decide which part of the work will be done by individual jobs in the array.

Read the documentation: [Slurm Workload Manager - Job Array Support](#)

Modules

- Managing available software is done through the system called “modules”.
- The software includes: scientific software, libraries (e.g. sqlite, cuda), core utilities (compiles, math libraries)
- The “module” command is used to work with the system, where subcommands include:
 - avail – see the list of available modules
 - **spider – search for a module and provide information about modules found**
 - load – to load a module
 - list – to list loaded modules
 - purge – clean your environment from any loaded modules
 - help – consult the manual
- Some examples:

```
$ module avail
( ..This will list all modules.. )
$ module load plumed
( ..This will load the plumed module.. )
$ module purge
( ..This will list the unloaded modules.. )
```
- What does the module command actually do? It doesn’t install the software, it adjusts the environment, so the application becomes available as it would be installed in a usual way. You can think of a module as a preset of **PATH**, **LD_LIBRARY_PATH** and other important environment variables.
- Load modules in the job script!

Exercise

1. Create a “hello world” batch job which will:
 - a. Get information about CPU (hint: use **lscpu** or similar command)
 - b. Report how many cores are available for the job (there are multiple ways to do this)
2. Create an array job in which each job prints out a **n-th** line of the same file, where **n** is the job’s task id (available through an environment variable `SLURM_ARRAY_TASK_ID`). Use [file name formatting](#) to create separate output files.

Assignment

(8p) Use an array job to render an animation from the Blender demo-files.

Warning! Rendering on a cluster is fast, but queue times are, in some cases, unpredictable. Please account for queue times from minutes to, in some cases, hours!

Detailed instructions:

- 1) (6p) Submit a job which uses an animation from the Blender [demo page](#), for example [Repeat Zone – Flower](#) (frame range 1-100). Not all demo scenes work in an environment without display!
 - a) Include all parameters in the job script.
 - b) Use the “plgrid” partition.
 - c) Assuming you chose to use the linked demo scene: using a batch job configuration of **1 node with 4 CPUs per task** can be a good choice, as rendering one frame may take up to 20 minutes. Requesting more CPUs will shorten render time, but queue times might increase. You can declare that each job will use up to 1GB of memory instead of the default 4GB per CPU – this might help with queue times.
 - i) (note) In a real-world scenario, choosing job configuration, accounting for application performance, and queue times is one of the main challenges.
 - d) Blender is available through the modules system described above.
 - e) Each job should render **one frame** which can be achieved with Blender in the following way: https://docs.blender.org/manual/en/latest/advanced/command_line/render.html
 - i) (hint) Ensure that the **-f** parameter is at the end of a command line! Blender tends to ignore it otherwise.
 - f) Verify if the images were rendered and if the animation looks OK.
- 2) (1p) Provide a part of **hpc-jobs-history** command output with information about your jobs. What efficiency was achieved?
- 3) (1p) Estimate how many CPU-hours were used for the whole animation.
 - a) (hint) This can be estimated based on job parameters and/or read from the **hpc-jobs-history** output.

Submission

Submit the job script, job outputs and answers to questions.